TechMedic v1.0 Technical Specification

AI-Assisted Technical Troubleshooting and Diagnostic System

Version: 1.0
Core architecture: User -> App -> LLM -> Jev -> LLM -> App -> User

1. Project Overview

TechMedic is an AI-assisted troubleshooting application for computer, software, network, peripheral, and consumer-electronics problems. It separates language understanding and explanation from constrained probabilistic decision-making.

The first LLM call interprets the user's report, extracts evidence, updates a structured diagnostic state, generates a small set of hypotheses, and proposes low-risk diagnostic tests. Jev then evaluates that state with typed decision questions. A second LLM call interprets the Jev result, selects the next user-facing step, explains why it is useful, and respects the safety screen.

The diagnostic process repeats as new evidence is collected.

2. Primary Goal

TechMedic should behave like a structured troubleshooting assistant rather than a generic chatbot. Its diagnostic cycle is:

Observe -> Form hypotheses -> Test -> Collect evidence -> Reassess -> Recommend

A good response should prefer one discriminating next step over a long list of loosely related suggestions.

3. Core Workflow

```
USER
  |
  v
TECHMEDIC APP
  |
  v
LLM #1 - Diagnostic Analyst
  - Understand problem
  - Extract facts
  - Preserve prior evidence
  - Create/update hypotheses
  - Propose candidate tests
  - Build structured state
  |
  v
```

```
DETERMINISTIC SAFETY SCREEN
  |
  v
JEV DECISION API
  - Choice: likely fault domain
  - Noul: evidence sufficient?
  - Noul: hardware failure likely?
  - Noul: professional service needed?
  - Score: urgency
  - Choice: next action
  |
  v
LLM #2 - Diagnostic Responder
  - Interpret Jev decisions
  - Select one next step
  - Explain why it matters
  - Request specific evidence
  - Respect safety warnings/blocks
  |
  v
SQLITE + STREAMLIT UI
  |
  v
USER
```

LLM #1 and LLM #2 may be separate calls to the same OpenAI model with different instructions.

4. Division of Responsibilities

4.1 LLM Responsibilities

The LLM handles tasks requiring language understanding and technical synthesis:

- Understand free-form problem descriptions.
- Extract hardware and software information.
- Identify symptoms, observations, and prior tests.
- Identify missing information.
- Generate a limited set of plausible hypotheses.
- Maintain structured diagnostic state across turns.
- Propose low-risk, discriminating tests.
- Interpret Jev results.
- Explain technical concepts and next steps in ordinary language.
- Summarize the likely diagnosis when evidence is sufficient.

The LLM should not invent diagnostic confidence percentages when Jev provides the relevant probability or confidence.

4.2 Jev Responsibilities

Jev handles bounded diagnostic judgments using typed questions:

- `choice` selects one option from a defined answer space and returns probabilities/confidence.
- `score` rates the state on an ordered scale and returns a score/distribution.
- `noul` returns a calibrated 0-1 probability for a yes/no judgment.

TechMedic v1.0 asks Jev six questions per completed diagnostic turn:

1. Likely fault domain (`choice`).
2. Whether evidence is sufficient for a likely root cause (`noul`).
3. Whether physical hardware failure is likely (`noul`).
4. Whether professional service is needed (`noul`).
5. Urgency (`score`).
6. Recommended next action (`choice`).

5. Jev Endpoint Selection

TechMedic v1.0 supports both Jev access routes.

Hosted self-service key

Keys beginning with `jv_live_` are routed automatically to:

```
POST https://jevtypesafeai.com/api/v1/decide
```

Official TypeSafe key

Other TypeSafe API keys are routed automatically to:

```
POST https://api.typesafe.ai/v1/systemone
```

Explicit override

An administrator can override automatic routing in `.env`:

```
JEV_API_URL=https://example.gateway/jev
```

When `JEV_API_URL` is blank, TechMedic chooses the route from the API-key prefix.

Authentication uses a Bearer token:

```
Authorization: Bearer <JEV_API_KEY>
Content-Type: application/json
```

The default Jev model is:

```
jev-latest
```

6. Example Diagnostic State


```
{
  "case_summary": "PyTorch cannot use the GPU",
  "device": {
    "type": "Windows PC",
    "manufacturer": "",
    "model": "",
    "operating_system": "Windows 11",
    "application": "PyTorch"
  },
  "symptoms": [
    "GPU appears in Windows",
    "PyTorch reports GPU unavailable"
  ],
  "observations": [],
  "tests_performed": [],
  "hypotheses": [
    {
      "name": "Incorrect PyTorch build",
      "rationale": "A CPU-only or incompatible build can import successfully while GPU suppo
    },
    {
      "name": "Driver/runtime problem",
      "rationale": "The operating system may see the GPU while the compute runtime is unavai
    }
  ],
  "missing_information": [],
  "candidate_next_tests": [],
  "possible_resolution": "",
  "status": "investigating"
}
```


7. Diagnostic Loop

Stage 1 - Problem Intake

The user describes the problem naturally and may optionally provide device type, manufacturer, model, and operating system.

Stage 2 - LLM Analysis

The diagnostic analyst extracts relevant facts, updates prior evidence, creates hypotheses, and proposes up to three candidate tests.

Stage 3 - Safety Screen

Candidate tests are screened for clearly hazardous or destructive procedures. High-voltage work, opening power supplies, damaging lithium batteries, and defeating safety interlocks are blocked in v1.0. Destructive or firmware/BIOS actions trigger warnings.

Stage 4 - Jev Evaluation

Jev receives the structured state plus the deterministic safety-screen result and returns typed decisions.

Stage 5 - LLM Response

The diagnostic responder receives the updated diagnostic state, Jev result, and safety result. It selects one practical next step, explains why it is useful, and tells the user exactly what result to send back.

Stage 6 - Persistence

The turn is stored in SQLite, including user text, diagnostic state, Jev result, safety result, and final assistant response.

Stage 7 - Reassessment

The cycle repeats until the issue is solved, a likely diagnosis is supported, more evidence is required, or professional service is appropriate.

8. v1.0 Troubleshooting Scope

Supported

Computers

- Windows startup problems.
- Performance problems.
- Application errors.
- Driver problems.
- GPU problems.
- Storage issues.
- Peripheral problems.

Networking

- Wi-Fi connectivity.
- Internet connectivity.
- DNS.
- Router problems.
- Basic LAN issues.

Software

- Installation failures.
- Dependency issues.
- Python problems.
- Application configuration.

Consumer electronics

- Television problems.
- Streaming-device problems.
- Basic display/audio problems.

Deferred / Out of Scope for v1.0

- Automotive diagnostics.
- Medical devices.
- Industrial machinery.
- High-voltage electrical systems.
- Safety-critical infrastructure.
- Advanced electronics board repair.

9. User Interface

TechMedic uses Streamlit for a local browser interface.

New Case Screen

- Large problem-description field.
- Optional device type.
- Optional manufacturer.
- Optional model.
- Optional operating system.
- Start Diagnosis button.

Diagnostic Screen

The diagnostic screen presents:

- Likely fault domain and Jev choice confidence.
- Probability that evidence is sufficient.
- Probability that hardware failure is likely.
- Urgency score.
- Probability that professional service is needed.
- Jev-recommended next action and choice confidence.
- Safety warnings or blocks.
- Conversation history.
- A chat input for the next test result or answer.

Developer Trace

A developer expander exposes:

- Structured diagnostic state.
- Raw Jev response.
- Safety-screen result.

This makes the LLM -> Jev -> LLM architecture auditable during development and demonstrations.

10. Data Storage

TechMedic uses SQLite. The default database path is:

`data/techmedic.db`

The database stores case metadata and every completed diagnostic turn. API keys are not stored in SQLite.

11. Configuration

TechMedic reads configuration from a local `.env` file.

```
OPENAI_API_KEY=
JEV_API_KEY=
OPENAI_MODEL=gpt-6-luna
JEV_MODEL=jev-latest
JEV_API_URL=
```

`JEV_API_URL` should normally remain blank so TechMedic can select the correct endpoint automatically.

12. Technology Stack

Component	Technology
Application language	Python
User interface	Streamlit
Language model	OpenAI Responses API
Decision engine	Jev
HTTP client	Requests
Persistence	SQLite
Configuration	python-dotenv / environment variables

13. Proposed Project Structure

```
TechMedic_v1_0/
|
```

```
|-- app.py
|-- requirements.txt
|-- .env.example
|-- .gitignore
|-- setup_windows.bat
|-- run_techmedic.bat
|
|-- techmedic/
|   |-- __init__.py
|   |-- config.py
|   |-- database.py
|   |-- diagnostic_engine.py
|   |-- jev.py
|   |-- llm.py
|   |-- models.py
|   |-- prompts.py
|   `-- safety.py
|
|-- data/
|   `-- .gitkeep
|
|-- tests/
|   |-- test_database.py
|   |-- test_jev_payload.py
|   |-- test_models.py
|   `-- test_safety.py
|
`-- docs/
    |-- ARCHITECTURE.md
    `-- TechMedic_v1_Technical_Specification.pdf
```

14. Minimum Viable Product Requirements

TechMedic v1.0 is complete when it demonstrates all of the following:

1. The user can describe a technical problem.
2. The first LLM call converts that description into structured diagnostic state.
3. Jev evaluates that state with typed questions.
4. The second LLM call uses Jev's output to determine and explain the next step.
5. The process can repeat as the user supplies evidence.
6. Case history persists in SQLite.
7. Safety warnings/blocks prevent clearly hazardous instructions.
8. The developer trace visibly demonstrates the LLM -> Jev -> LLM flow.

15. Initial Validation Scenarios

Scenario 1 - Python CPU/GPU Build

Symptom: A GPU exists and is detected by Windows, but PyTorch cannot use it.

Expected path: determine the installed PyTorch build and compute-runtime compatibility before concluding hardware failure.

Scenario 2 - Internet/DNS Failure

Symptom: The computer connects to Wi-Fi and can reach an IP address, but normal websites do not load.

Expected path: use a discriminating DNS test and identify DNS configuration as a likely cause when supported.

Scenario 3 - Display/Backlight Problem

Symptom: Television audio continues but the picture disappears.

Expected path: collect evidence that distinguishes input/source problems, video-processing issues, and display/backlight/panel-system faults without recommending hazardous internal work.

16. Safety Requirements

TechMedic must not blindly recommend dangerous procedures. In v1.0 it should block or avoid instructions involving:

- Exposed mains/high voltage.
- Opening power supplies.
- Opening or damaging lithium batteries.
- Defeating safety interlocks.
- Other clearly hazardous service procedures.

It should warn before actions involving:

- Data destruction.
- Disk formatting or partition changes.
- Firmware flashing.
- BIOS/UEFI modification.
- Registry modification.
- Security-sensitive configuration.

When a safe external diagnostic step cannot reasonably continue, TechMedic should recommend qualified professional service.

17. Success Criteria

- The full User -> App -> LLM -> Jev -> LLM -> App -> User workflow is operational.
- Jev materially influences the diagnostic next step.
- Diagnostic state persists between turns.
- Jev probabilities/confidences are shown without being replaced by LLM-invented percentages.
- At least three controlled diagnostic scenarios can be demonstrated.
- The application can use a hosted `jv_live_` key without manual source-code changes.
- The endpoint can be overridden in configuration for future gateways or official TypeSafe access.
- The architecture can be extended to additional diagnostic domains.

18. Future Development

v1.1

- Save/load improvements.
- Exportable diagnostic reports.
- Improved case history and search.

v1.2

- Screenshot/image upload.
- Error-message extraction from screenshots.
- Log-file upload and analysis.

v1.3

- Current web research for drivers, firmware, release notes, and manufacturer documentation.

v1.4

- Manufacturer-specific troubleshooting modules.

v2.0

- Retrieval from service manuals and technical documentation.
- Hardware inventory ingestion.
- Remote diagnostic scripts with explicit safety and authorization controls.
- Automatic system-information collection.

19. Guiding Principle

TechMedic should behave less like:

> Here are ten things you could try.

and more like:

> Based on the evidence so far, these are the most likely causes. Perform this specific test next because its result will help distinguish between them.

That distinction is the core design principle of TechMedic.

20. Reference Documentation

- Jev hosted API documentation: https://www.jevtypesafeai.com/typesafe/docs
- Jev hosted decision endpoint: https://jevtypesafeai.com/api/v1/decide
- TypeSafe official API: https://api.typesafe.ai/docs
- OpenAI API documentation: https://platform.openai.com/docs
- Streamlit documentation: https://docs.streamlit.io